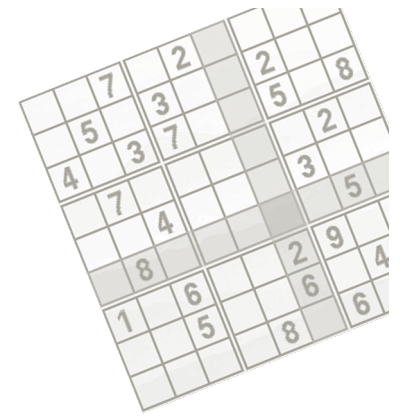




Constraint Programming

Roman Barták

Department of Theoretical Computer Science and Mathematical Logic

Global Constraints

Can we achieve GAC **faster than a general GAC algorithm**?

- for example revision of $A < B$ can be done much faster via bounds consistency.

Can we write a **filtering algorithm for a constraint** whose **arity varies**?

- for example all_different constraint

We can exploit **semantics of the constraint** for efficient filtering algorithms that can work with any number of variables.

👉 **global constraints** 👉

↳ arity never fixed

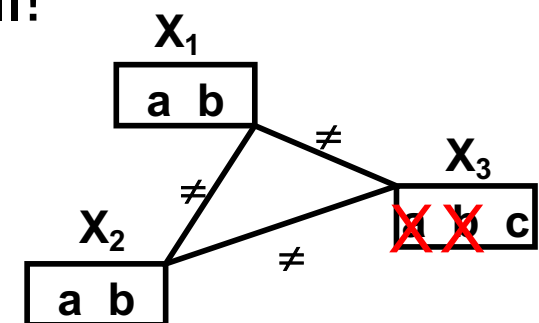
Logic-based puzzle, whose goal is to enter digits 1-9 in cells of 9×9 table in such a way, that no digit appears twice or more in every row, column, and 3×3 sub-grid.

9	6	3	1	7	4	2	5	8
1	7	8	3	2	5	6	4	9
2	5	4	6	8	9	7	3	1
8	2	1	4	3	7	5	9	6
4	9	6	8	5	2	3	1	7
7	3	5	9	6	1	8	2	4
5	8	9	7	1	3	4	6	2
3	1	7	2	4	6	9	8	5
6	4	2	5	9	8	1	7	3

How to model such a problem?

- variables **describe the cells**
- **inequality constraint** connect each pair of variables in each row, column, and sub-grid
- Such constraints do not propagate well!

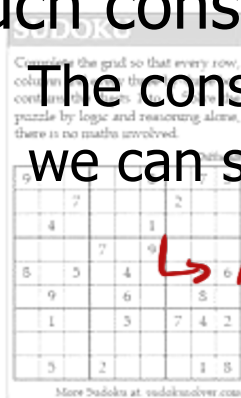
- The constraint network is AC, but
- we can still remove some values.



→ AC je na nerovnost
dodatna ass

recognised puzzle arrived in Japan in the late 1970s by the magazine 'Math Logic Puzzles' under the title 'Number Squares'. It is thought that this type of puzzle helps prevent the advance of Alzheimer's and offers some comfort for short-term illness. Through its therapeutic thought processes, Sudoku has no requirement to be either symmetrical or called Latin Squares. Some people may argue that some approach has advantages for mathematicians, but differences in the way each type of Sudoku are 'carries magiques'.

Sudoku with no letter, we believe that Sudoku is better when they are square. The identification of patterns draws to create challenging and aspect of the brain. Unlike a challenge to solve, rendering any layout obsolete.



More Sudokus at: vudokusolver.com

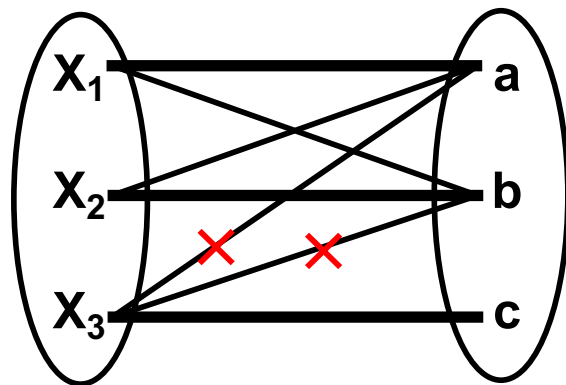


This constraint models a complete set of binary inequalities.

$$\text{all_different}(\{X_1, \dots, X_k\}) = \{(d_1, \dots, d_k) \mid \forall i \ d_i \in D_i \ \& \ \forall i \neq j \ d_i \neq d_j\}$$

Domain filtering is based on **matching in bipartite graphs**

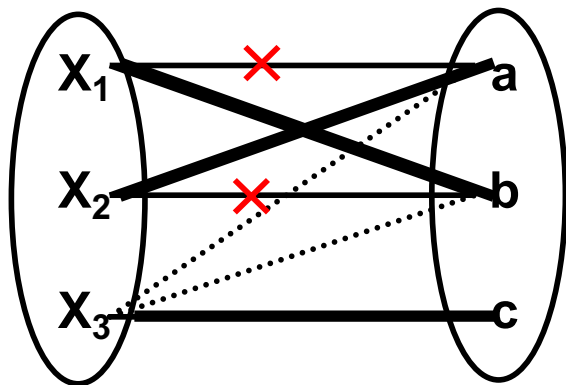
(nodes = variables+values, edges = description of domains)



Initialization:

→ size = n, jinak fail

- 1) find a maximum matching
- 2) remove all edges that do not belong to any maximum matching

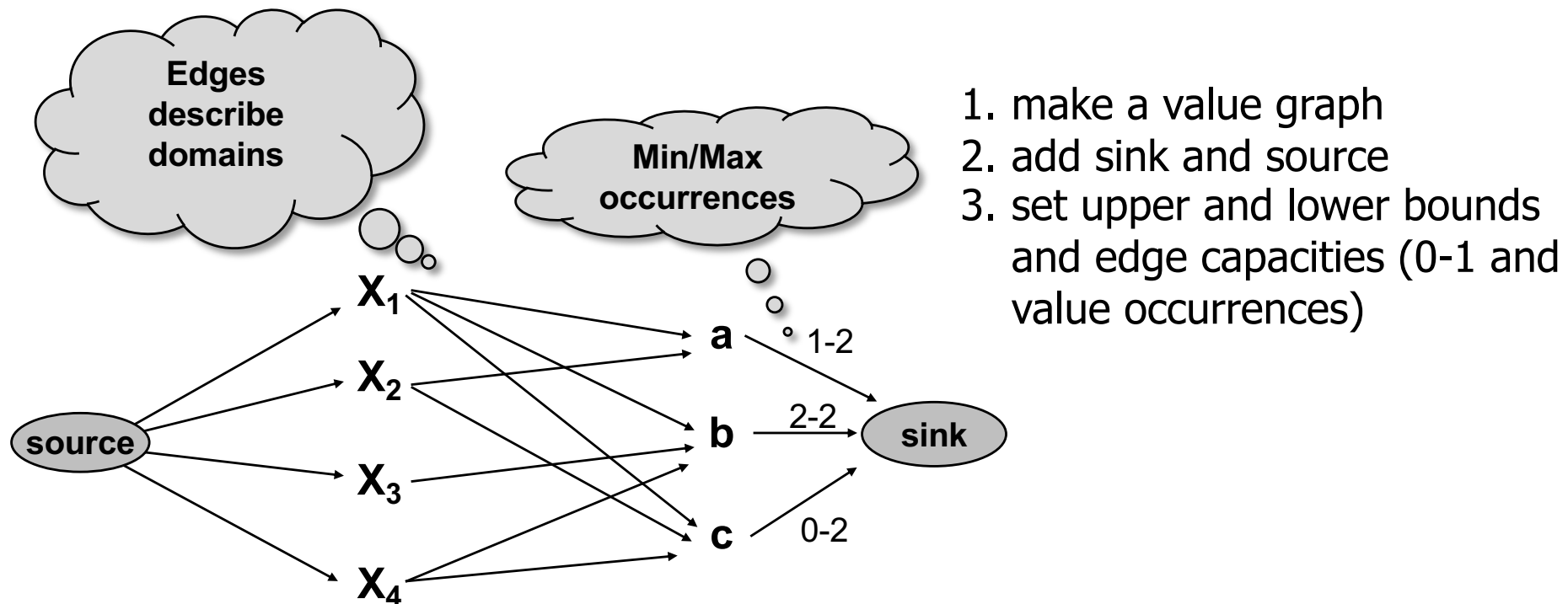


Incremental propagation ($X_1 \neq a$):

- 1) remove "deleted" edges
- 2) find a new maximum matching
- 3) remove all edges that do not belong to any maximum matching

→ smažu i to, co může vyrobiť matching

- A generalization of all-different
 - the number of occurrences of a value in a set of variables is restricted by minimal and maximal numbers of occurrences
- Efficient filtering is based on **network flows**.



A maximal flow corresponds to a feasible assignment of variables! We will find edges with zero flow in each maximal flow and then we will remove the corresponding edges.

Existence of **symmetrical solutions** decreases efficiency of constraint satisfaction (symmetrical search spaces are explored).

A classical example with many symmetries – **sports tournament scheduling**.

- there are n teams
- each team plays with all other teams, i.e., $(n-1)$ rounds
- each team plays as a home team or a guest team

How to model such a problem?

- Round i is modelled by a sequence of **match codes** K_i .
 - $K_{i,j}$ is a code of j -th match at round i
- We can swap matches at each round – **match symmetry**.
 - match symmetry is removed by constraint $K_{i,j} < K_{i,j+1}$ *→ ordering*
- We can swap complete rounds – **round symmetry**.
 - round symmetry is removed by constraint $K_i <_{\text{lex}} K_{i+1}$.



$$\mathbf{lex}(\{X_1, \dots, X_n\}, \{Y_1, \dots, Y_n\}) \equiv (X_1 \leq Y_1) \wedge (X_1 = Y_1 \Rightarrow X_2 \leq Y_2) \wedge \dots \wedge (X_1 = Y_1 \wedge \dots \wedge X_{n-1} = Y_{n-1} \Rightarrow X_n < Y_n)$$

- α : the variables before α are all instantiated and pairwise equal
- β : vectors starting at β are lexicographically ordered but “oppositely”
 $\text{floor}(\{X_\beta, \dots, X_n\}) >_{\text{lex}} \text{ceiling}(\{Y_\beta, \dots, Y_n\})$

$X = \langle \{2\}, \{1\}, \{1,2,3,4,5\}, \{1,2\}, \{3,4,5\} \rangle$ change X_2 so at least $X_2 = Y_2$
 $Y = \langle \{2\}, \{1\}, \{0,1,2,3,4\}, \{0,1\}, \{0,1,2\} \rangle$ and again shift pointer α

$X = \langle \{2\}, \{1\}, \{1,2,3\}, \{1,2\}, \{3,4,5\} \rangle$ because $\alpha = \beta - 1$
 $Y = \langle \{2\}, \{1\}, \{2,3,4\}, \{0,1\}, \{0,1,2\} \rangle$ force constraint $X_\alpha < Y_\alpha$
 $\alpha \uparrow \quad \uparrow \beta$





Rostering

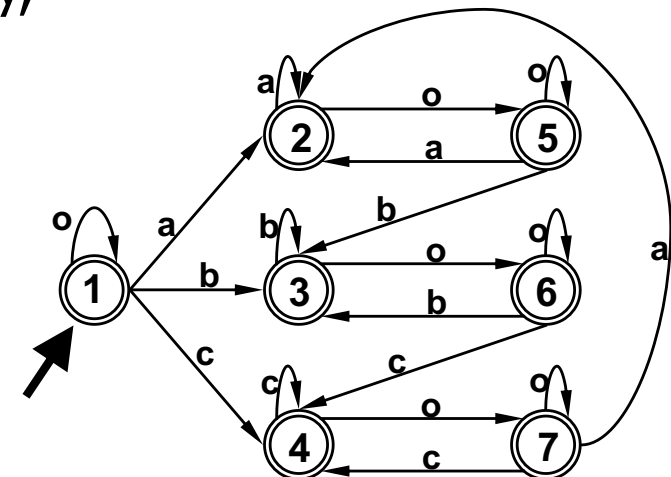
- **scheduling of shifts**, for example in hospitals
- There are typically specific shift sequencing constraints (given by trade unions, law etc.)

Example:

- shifts: a, b, c, o (o means a free shift)
- constraints:
 - the same shift can repeat each day
 - at least one o shift is between a, b, between b, c, and between c, a
 - a-o*-c, b-o*-a, c-o*-b are not allowed (o* is a sequence of o shifts)
- Any shift can be used the first day, only shifts b, o can be used the second day, shifts a, c, o for the third day, shifts a, b, o for the fourth day, and shift a the fifth day.

How to model such a problem?

- variables describe shifts in days
- And what about constraints?
 - using a **finite state automaton** (FSA)



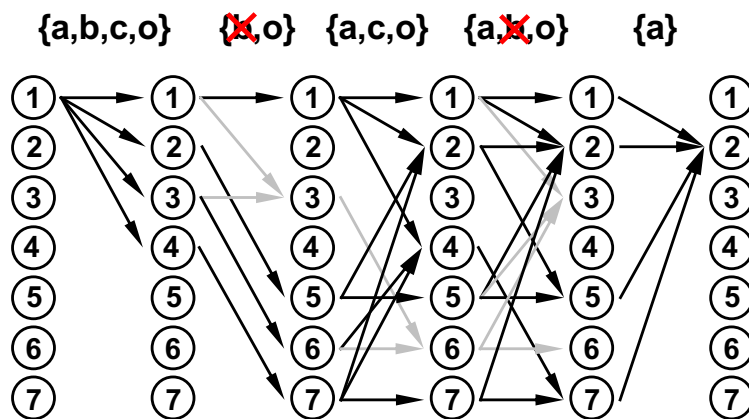


models a sequence of symbols **accepted by a FSA**

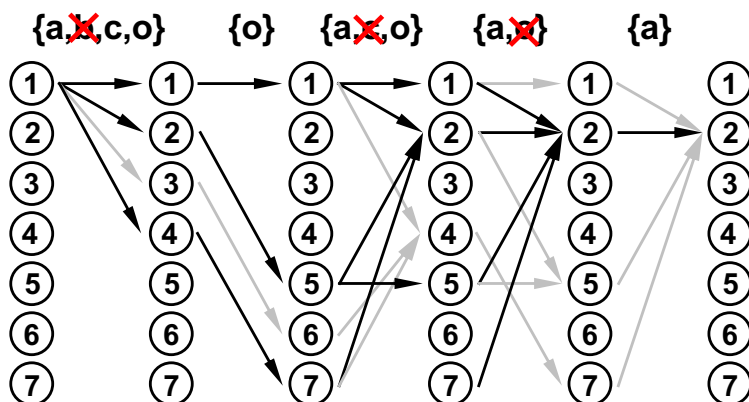
$$\text{regular}(A, \{X_1, \dots, X_k\}) = \{(d_1, \dots, d_k) \mid \forall i \ d_i \in D_i \wedge d_1 \dots d_k \in L(A)\}$$

filtering is based on representing all possible computations of a FSA using a **layered directed graph** (layer=states, arc=transitions)

Initialisation



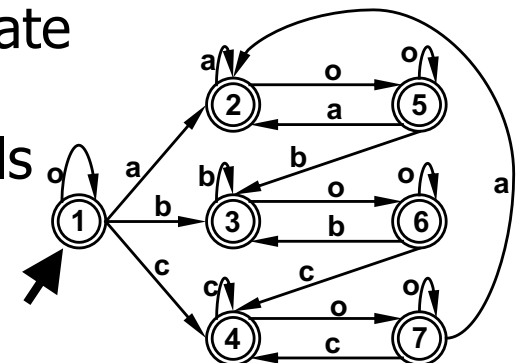
1. add arcs going from the initial state based on the symbols in the variables' domains
2. during the backward run, remove the arcs that are not on paths to the final states
3. remove the symbols without any arc



Incremental filtering ($X_4 \neq o$):

1. remove arcs for the deleted symbol
2. propagate the update in both directions
3. remove the symbols without any arc

*→ když je něco deleted
= cloning*





Can we also model a sequence of symbols defined by a **context-free grammar**?

$\text{grammar}(G, \{X_1, \dots, X_k\}) = \{(d_1, \dots, d_k) \mid \forall i \ d_i \in D_i \wedge d_1 \dots d_k \in L(G)\}$

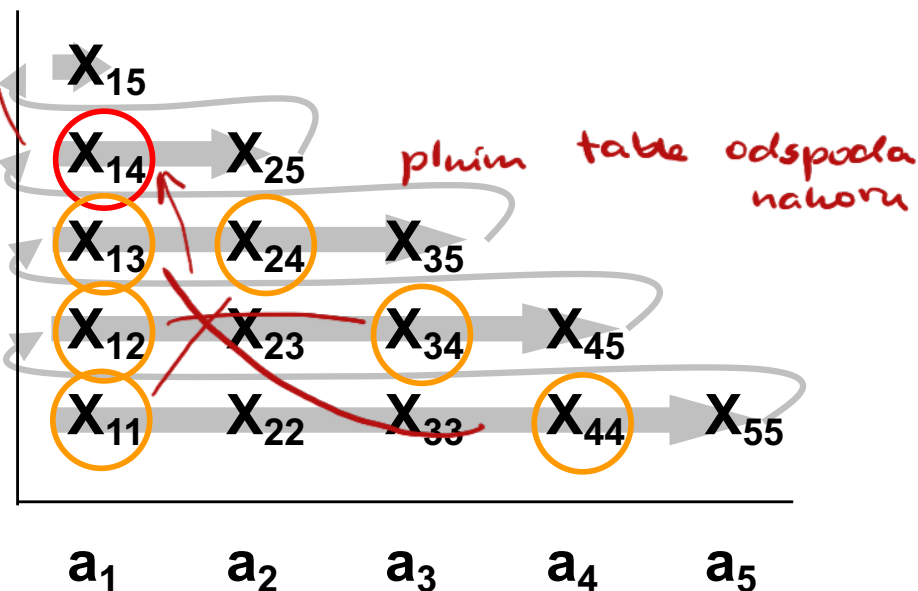
filtering is based on **algorithm CYK** for a Chomsky NF ($A \rightarrow BC \mid x$)

idea: $X_{i,j} = \{A \mid A \Rightarrow^* a_i a_{i+1} \dots a_j\}$ *→ pro slovo a gramatiku v Ch. norm. f. řekne, jestli to slovo do gram. patří*

start with: $X_{i,i} = \{A \mid (A \rightarrow a_i) \in P\}$

continue: $X_{i,j} = \{A \mid \exists k: i \leq k < j \ B \in X_{i,k} \wedge C \in X_{k+1,j} \wedge (A \rightarrow BC) \in P\}$

if $S \in X_{1,n}$, the $a_1 a_2 \dots a_n$ belongs to the language



$S \rightarrow AB \mid BC$ $A \rightarrow BA \mid a$ $B \rightarrow CC \mid b$ $C \rightarrow AB \mid a$				
{S,A,C}				
-	{S,A,C}		{B}	
-	{B}		{B}	
{S,A}	{B}	{S,C}	{S,A}	
{B}	{A,C}	{A,C}	{B}	{A,C}
b	a	a	b	a

Ch.n.s → 2 neterminally ne 1 terminal

Let us go back to the **regular** constraint, which behaves like **sliding a special transition constraint over a sequence of variables**.

Such a principle can be generalized!

$\text{slide}_j(C, \{X_1, \dots, X_n\}) \equiv \forall i \ C(X_{ij+1}, \dots, X_{ij+k})$ *j = jump*

- C is a k-ary constraint
- constant j determines the slide length

Some examples:

- $\text{regular}(A, \{X_1, \dots, X_n\}) \equiv \text{slide}_2(C, \{Q_0, X_1, Q_1, \dots, X_n, Q_n\})$
 $C(P, X, Q)$ represents a transition $\delta(P, X) = Q$, $Q_0 = \{q_0\}$, $Q_n = F$ *role of a 3 pointers*
- $\text{lex}(\{X_1, \dots, X_n\}, \{Y_1, \dots, Y_n\}) \equiv \text{slide}_3(C, \{B_0, X_1, Y_1, B_1, \dots, X_n, Y_n, B_n\})$
 $C(B, X, Y, C) \equiv B=C=1$ or $(B=C=0$ and $X=Y)$ or $(B=0, C=1$ and $X<Y)$
 $B_0 = 0$, $B_n = 1$ (strict lex), B_n in $\{0, 1\}$ (non lex)
- $\text{stretch}(\{X_1, \dots, X_n\}, s, l, t) \equiv \text{slide}_2(C, \{X_1, S_1, \dots, X_n, S_n\})$ *vectors of numbers → binary*
 $C(X_i, S_i, X_{i+1}, S_{i+1}) \equiv X_i = X_{i+1}, S_{i+1} = 1 + S_i, S_{i+1} \leq l(X_i),$
or $X_i \neq X_{i+1}, S_i \geq s(X_i), S_{i+1} = 1, (X_i, X_{i+1}) \in t$ *→ pro rostering banger*
 $S_1 = 1$ *podobné jako regular s counters*
t = ?

...

l = upper-bound → kolikrát můžeme být v daném stavu
s = lower-bound → kolikrát musíme být v daném stavu (za sebou než můžeme)
S_i = counter opakování X_i

A **scheduling problem** is a problem of allocating a known set of activities to available resources and time.

We will assume a unary resource now.

Unary resource allows allocation of at most one activity to the resource at any time.

fixed duration and time window for each activity is given

How to model such a problem?

- **variable start(A)** describes the start time of activity A
- **constraints** ensure that no activities overlap in time

$$(\text{start(A)} + \overset{\text{processing time}}{p(A)} \leq \text{start(B)}) \vee (\text{start(B)} + p(B) \leq \text{start(A)})$$

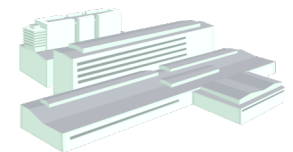
(hence also the name **disjunctive resource**)

- or other relations such as **precedence** $A \ll B$

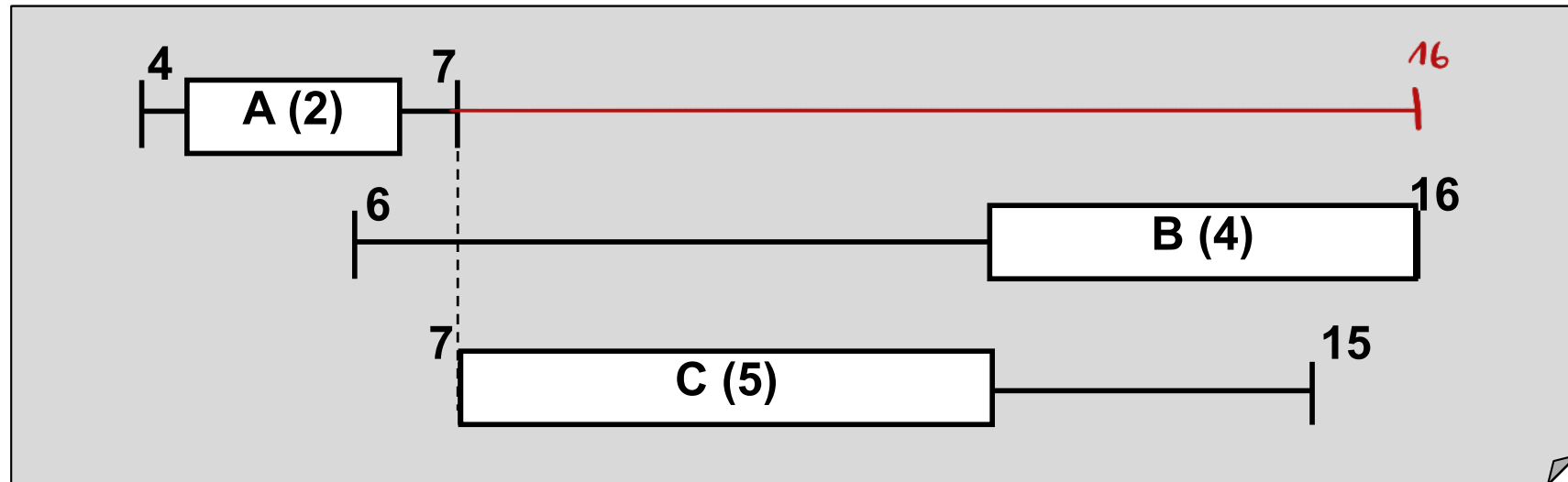
$$\text{start(A)} + p(A) \leq \text{start(B)}$$

Recall – disjunctions do almost no filtering!

! disj. constr. se špatně propagují (jako nerovnosti) ↑

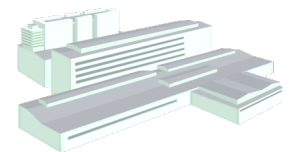


What does happen when A is not processed first?



There is not enough time to process A, B,C and A must be first!

restriktum čas na 7 → jinak nelze zkompletovat

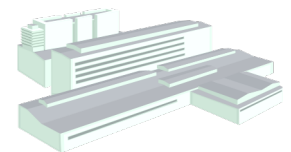


*shrinking time windows***Filtering rules:**

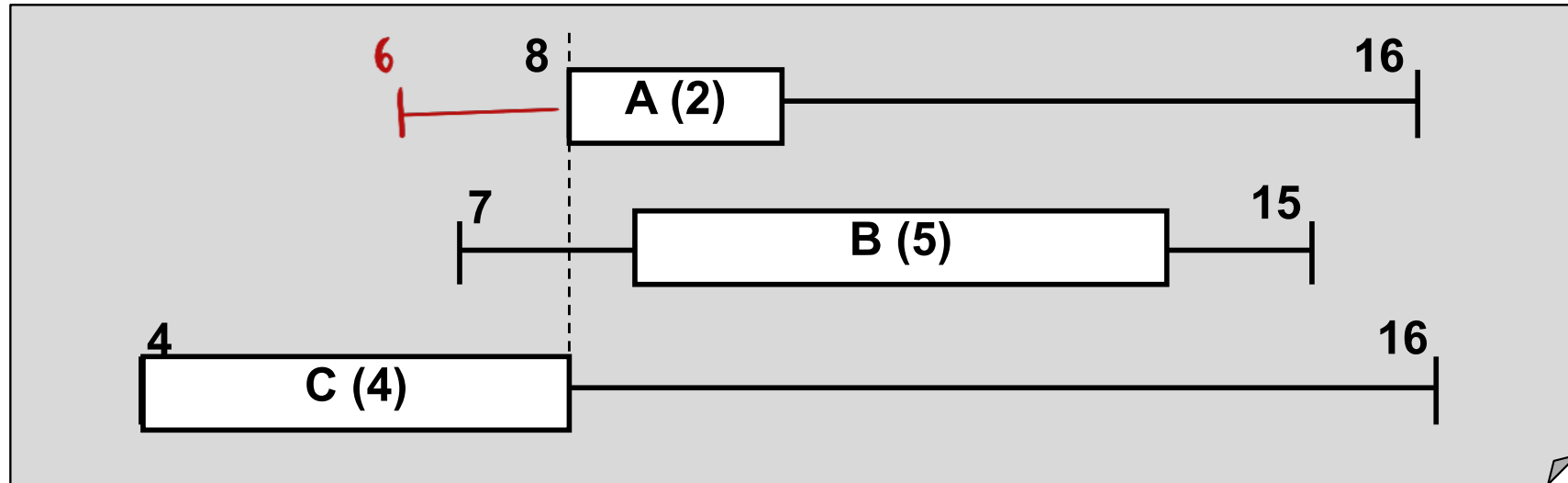
- $p(\Omega \cup \{A\}) > lct(\Omega \cup \{A\}) - est(\Omega) \Rightarrow A \ll \Omega$
- $p(\Omega \cup \{A\}) > lct(\Omega) - est(\Omega \cup \{A\}) \Rightarrow \Omega \ll A$
- $A \ll \Omega \Rightarrow end(A) \leq \min\{ lct(\Omega') - p(\Omega') \mid \Omega' \subseteq \Omega \}$
- $\Omega \ll A \Rightarrow start(A) \geq \max\{ est(\Omega') + p(\Omega') \mid \Omega' \subseteq \Omega \}$

*latest compl. time**earliest start time**set of activities***In practice:**

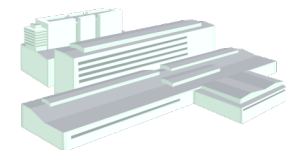
- we need to assume $n \cdot 2^n$ pairs (A, Ω) (to many!)
- Instead of sets Ω we can use **task intervals** $[X, Y]$
 $\{C \mid est(X) \leq est(C) \wedge lct(C) \leq lct(Y)\}$
 $\hookrightarrow$ time complexity $O(n^3)$, a frequently used incremental algorithm
- there are also algorithms with time complexity $O(n^2)$ and $O(n \cdot \log n)$



What does happen if A is processed first?



There is not enough time for B and C and hence A cannot be first



Rules for not_first:

$$p(\Omega \cup \{A\}) > \text{lct}(\Omega) - \text{est}(A) \Rightarrow \neg A \ll \Omega$$

$$\neg A \ll \Omega \Rightarrow \text{start}(A) \geq \min\{ \text{ect}(B) \mid B \in \Omega \}$$

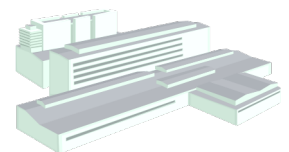
Rules for not_last:

$$p(\Omega \cup \{A\}) > \text{lct}(A) - \text{est}(\Omega) \Rightarrow \neg \Omega \ll A$$

$$\neg \Omega \ll A \Rightarrow \text{end}(A) \leq \max\{ \text{lst}(B) \mid B \in \Omega \}$$

In practice:

- can be implemented with time complexity $O(n^2)$ and space complexity $O(n)$





© 2013 Roman Barták

Department of Theoretical Computer Science and Mathematical Logic

bartak@ktiml.mff.cuni.cz